

Episign - Technical Summary

APPING2 - Programmation IOS

Robin Vidal, Marie-Lou Allain, Michael Rousseau, Naïm Chefirat

May 12, 2026

What is EpiSign?

EpiSign is an attendance-signing system that replaces QR-code proof-of-presence with **ultrasonic in-room validation**. A teacher emits a time-based one-time password (TOTP) over ultrasound (17–20 kHz, inaudible to humans) from a web page; students' iPhones listen, decode the code, and submit a signature to the backend. Manual 6-digit TOTP entry remains available as a universal fallback.

The project has three components:

- **iOS student app** (Swift/SwiftUI) — captures the ultrasonic signal via the microphone, decodes the TOTP, collects a handwritten signature via PencilKit, and submits attendance to Supabase.
- **Teacher web page** (TypeScript + Vite) — authenticates the teacher, fetches session details, generates the TOTP, and broadcasts it continuously via ggwave WASM over the browser's AudioContext.
- **Supabase backend** — PostgreSQL database with Row-Level Security, two Edge Functions (authentication, TOTP validation and signature recording), and object storage for signature PNGs.

iOS Student App

Architecture Overview

The app follows a straightforward SwiftUI + SwiftData architecture with a service layer. It targets iOS 17+ and requires Xcode 15+.

EpiSign/

EpiSignApp.swift # App entry point, auth gate, model container

Models/

Course.swift # SwiftData @Model - cached course info

Signature.swift # SwiftData @Model - signed attendance record

LocalSignatureDraft.swift # SwiftData @Model - offline retry queue

DeviceInfo.swift # SwiftData @Model - device binding UUID

Services/

AuthManager.swift # Supabase auth (magic link, password, deep link)

SupabaseManager.swift # Singleton Supabase client + config

CourseService.swift # Fetch courses/signatures, sync to SwiftData

SigningService.swift # Submit signature to Edge Function

AudioManager.swift # AVAudioEngine mic capture, ggwave decode, vDSP FFT

Views/

Auth/LoginView.swift # Magic link or password login

MainTabView.swift # Two-tab layout: Sign / Timetable

SignTab/

SignTabView.swift # Detects current course, shows signing UI

CourseCardView.swift # Course list item

StatCardView.swift # Attendance statistics card

Signing/

SigningView.swift # Core signing flow

SpectrumVisualizerView.swift # Circular FFT bar visualizer (Canvas)

AudioVisualizerView.swift # (auxiliary visualizer)

SignatureSheetView.swift # PencilKit canvas + submission

TimetableTab/

TimetableView.swift # Attendance stats + course list

Packages/

GGWave/ # Local Swift Package wrapping ggwave C sources

Package.swift

Sources/

C GGWave/ # C/C++ ggwave sources

GGWave/

GGWaveActor.swift # Swift wrapper

Key Dependencies (SPM)

Package	Purpose
supabase-swift	Auth, PostgREST queries, Edge Function invocations
SwiftData	Local persistence for courses, signatures, offline drafts, device ID
PencilKit	Apple Pencil signature capture (PKCanvasView via UIViewRepresentable)
GGWave (local)	Ultrasonic FSK encoder/decoder — C sources compiled as a Swift Package target, exposed through GGWaveDecoder

Auth Flow

AuthManager (an **@Observable** class) bootstraps on launch: it checks for an existing Supabase session, then listens to **authStateChanges** for sign-in/sign-out events. The app supports three login methods:

1. **Magic link** — **signInWithOtp** with a custom URL scheme (**episign://login-callback**) handled via **onOpenURL**.
2. **Sign up** — email + password via **supabase.auth.signUp**.
3. **Sign in** — email + password via **supabase.auth.signIn**.

On first authentication, **ensureStudentProfile** upserts a row into the **students** table with a generated **device_id** for device binding.

Signing Flow

1. **CourseService.syncToLocal** fetches all courses and the student's existing signatures from Supabase, then replaces the **SwiftData** store. This runs on app launch and on pull-to-refresh.
2. **SignTabView** checks **Course.isCurrent** to find a session whose time window [**startsAt**, **endsAt**] includes now. If found, it presents the **SigningView**.
3. **AudioManager** starts an **AVAudioEngine** input tap at hardware sample rate (typically 48 kHz). Each audio buffer is processed by two consumers:
 - **Spectrum visualizer** — **vDSP_DFT** produces 64 frequency bars restricted to the 17–21 kHz band, displayed in a circular **Canvas** layout in **SpectrumVisualizerView**.
 - **GGWave decoder** — samples are accumulated into 1024-frame chunks and fed to **GGWaveDecoder.decode()**. A valid 6-digit TOTP is published as **detectedTOTP**.
4. The detected TOTP auto-fills the manual entry field. The student taps **Signer**, which opens **SignatureSheetView** — a **PencilKit** canvas. After signing, the PNG is SHA-256 hashed, base64-encoded, and submitted to the **sign** Edge Function along with the TOTP, course ID, slot, device ID, and timestamp.
5. On network failure (**URLError**), the signature is saved locally as a **LocalSignatureDraft** and retried on next launch (via **retryOfflineDrafts** in **EpiSignApp**).

Screenshots

Dashboard

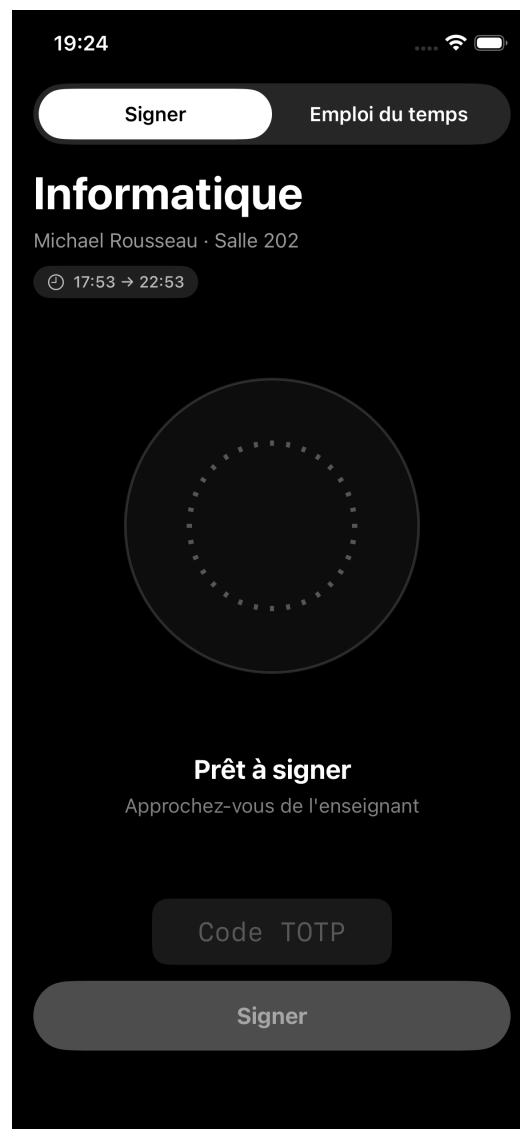
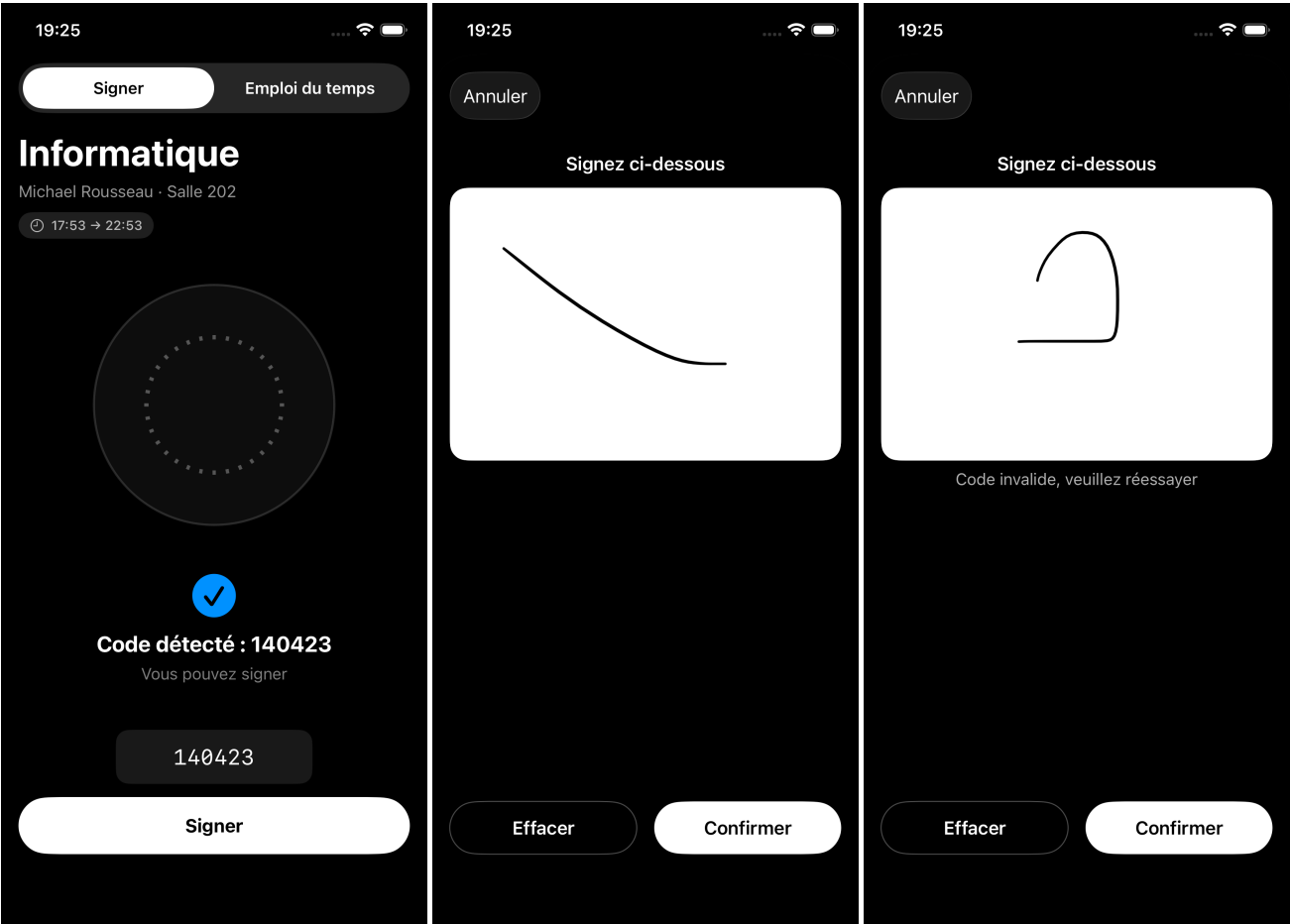
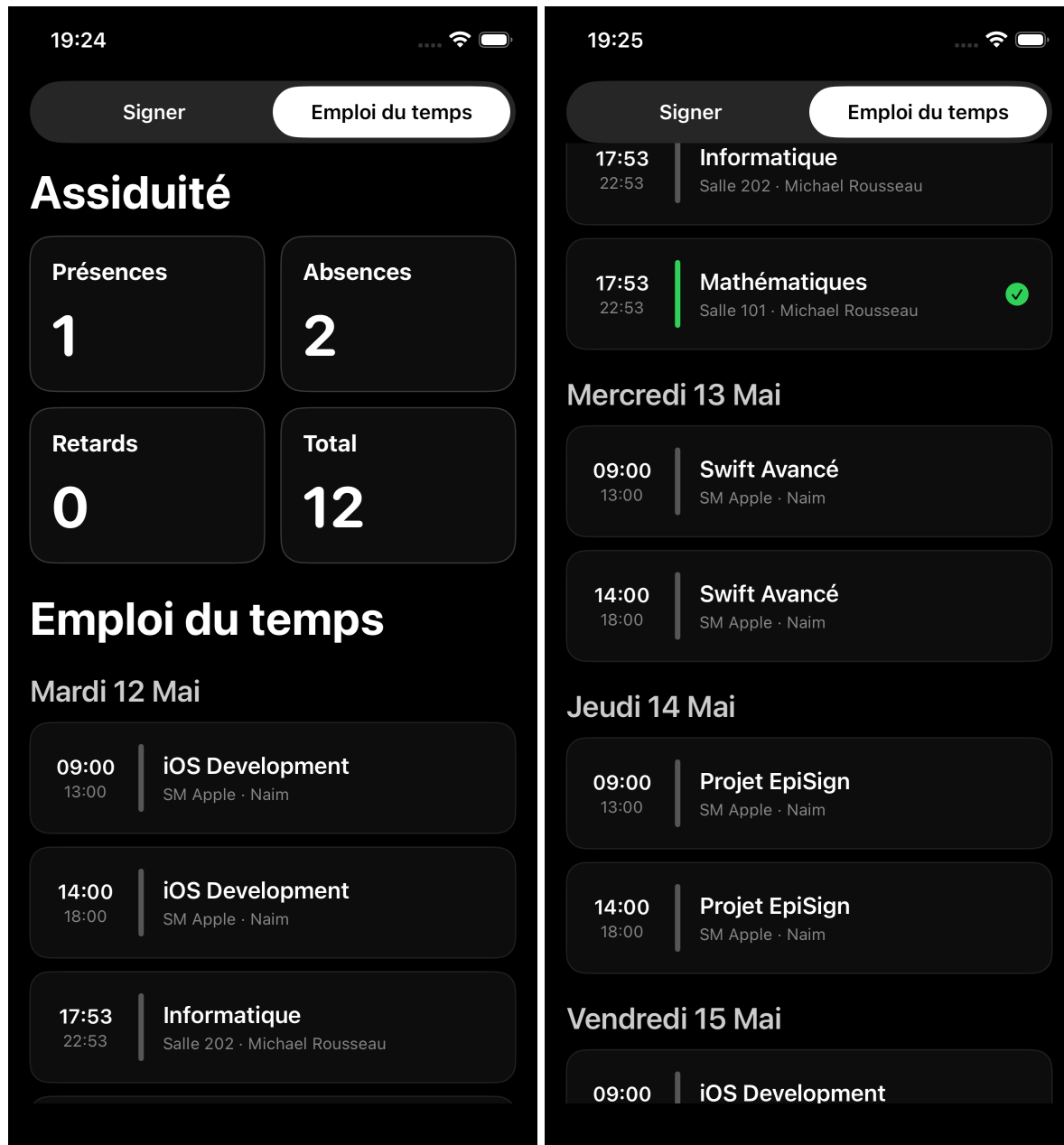


Figure 1: Login

Signing



Timetable



Supabase Backend

Database Schema

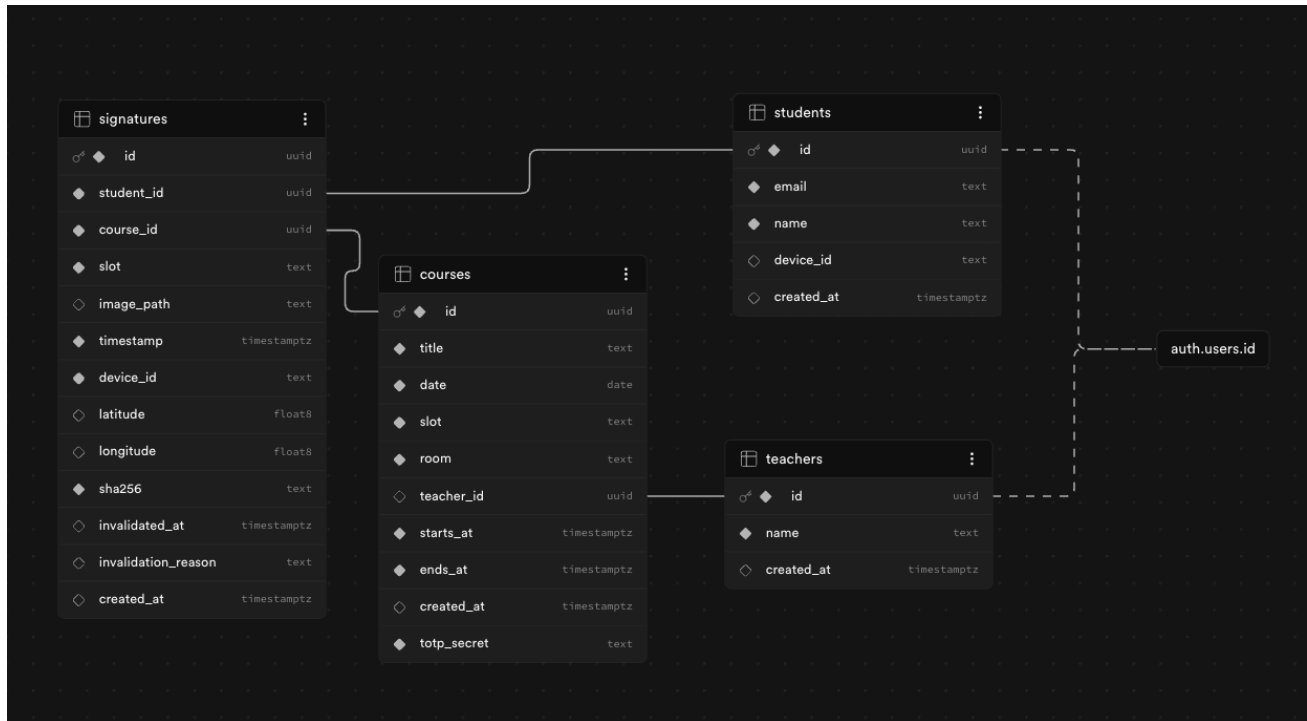


Figure 2: Schema

Four tables with Row-Level Security:

`students` (`id` UUID PK = `auth.uid()`, `email` TEXT, `name` TEXT, `device_id` TEXT)

`teachers` (`id` UUID PK = `auth.uid()`, `name` TEXT, `created_at` TIMESTAMPTZ)

`courses` (`id` UUID PK, `title` TEXT, `date` DATE, `slot` TEXT CHECK IN ('morning','afternoon'), `room` TEXT, `teacher_id` UUID FK→`teachers`, `starts_at` TIMESTAMPTZ, `ends_at` TIMESTAMPTZ, `totp_secret` TEXT)

`signatures` (`id` UUID PK, `student_id` UUID FK→`students`, `course_id` UUID FK→`courses`, `slot` TEXT, `image_path` TEXT, `timestamp` TIMESTAMPTZ, `device_id` TEXT, `latitude` DOUBLE PRECISION, `longitude` DOUBLE PRECISION, `sha256` TEXT, `invalidated_at` TIMESTAMPTZ NULL, `invalidation_reason` TEXT NULL, UNIQUE(`student_id`, `course_id`, `slot`))

Key design decisions:

- **totp_secret lives on courses**, not teachers. This means every course session produces a different TOTP code, even if two courses overlap in time.
- **teachers.id references auth.uid()** — teacher accounts are Supabase Auth users whose UUID matches the `teachers` row, enabling the Edge Function to verify ownership of a course.
- **RLS policies**: students can read/update their own row; courses are readable by any authenticated user; signatures are read-only for students (inserts go through Edge Functions). The

`teachers` table has no RLS policy for the `authenticated` role — only `service_role` can read it.

Edge Functions

Both are Deno/TypeScript functions deployed on Supabase:

sign — Called by the iOS app to submit attendance.

Validation steps (all run server-side with `service_role`):

1. Verify the student's JWT.
2. Check the course exists and current time is within `[starts_at, ends_at]`.
3. Validate the 6-digit TOTP against `courses.totp_secret` with a ± 30 s window (using the `otpauth` library).
4. Enforce uniqueness on `(student_id, course_id, slot)`.
5. Check device binding: if `students.device_id` is set, it must match the submitted `device_id`.
6. Upload the signature PNG to the `signatures` storage bucket.
7. Insert the `signatures` row.

teacher-session-key — Called by the teacher web app to obtain the TOTP secret for a given course.

1. Verify the teacher's JWT.
2. Fetch the course row using `service_role`.
3. Confirm `course.teacher_id` matches `user.id` (only the course's teacher can get the secret).
4. Return `{ totp_secret, course_title, starts_at, ends_at }`.

Storage

A single `signatures` bucket stores PNG files at `<student_id>/<signature_id>.png`, with RLS allowing students to read only their own files.

Teacher Web App

A vanilla Vite + TypeScript app (`teacher-web/`) that authenticates teachers and emits TOTP codes via ultrasonic audio.

Authentication

Uses Supabase `signInWithOtp` (magic link). In dev mode (`VITE_SUPABASE_SERVICE_ROLE_KEY` is set), the app bypasses email delivery by calling `supabase.auth.admin.generateLink` and displaying the magic link directly in the browser — avoiding the 2-emails-per-hour rate limit during testing.

Screen Flow

1. **Login** — teacher enters email, receives magic link (or gets it displayed in dev mode).
2. **Session list** — on auth, `fetchCourses(teacherId)` queries courses where `teacher_id` matches the current user. Displays upcoming sessions as cards.
3. **Emit** — selecting a course calls `teacher-session-key` to retrieve the TOTP secret, then:
 - Instantiates an `AudioContext` at 48 kHz.
 - Loads ggwave WASM from CDN, initializes an encoder instance.
 - Every 5 seconds, generates the current 6-digit TOTP (using the `otpauth` library), encodes it with ggwave's ultrasound protocol (17–20 kHz), and plays the waveform.
 - Displays the TOTP in large type as a manual fallback, with a 30-second countdown timer bar.
 - If ggwave fails, falls back to display-only mode (manual TOTP entry only).

Key Technical Choices

- **No framework** — the app uses vanilla TypeScript with manual DOM manipulation and state-based rendering. Screens are functions that return HTML strings; event listeners are attached after each render.
- **ggwave WASM** loaded dynamically from CDN at runtime. The waveform bytes are reinterpreted as `Float32Array` and played through a short-lived `AudioBufferSourceNode`.
- **OTPAuth UMD** is loaded via a script tag in `index.html` and accessed through `window.OTPAuth.TOTP`.